CS & IT ENGINEERING

Programming in C

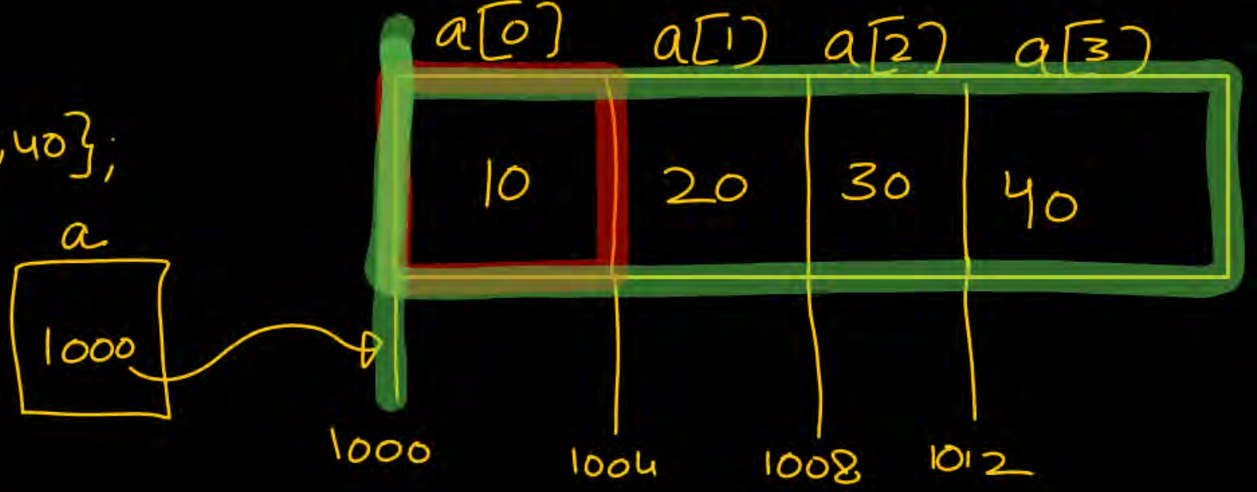
Arrays and Pointers

Lec- 02

By- Pankaj Sharma sir

Arrays-II

```
int a[4] = {10,20,30,40};
```



- `printf("%u", &a[0]);` 1000
- `printf("%u", &a);` 1000

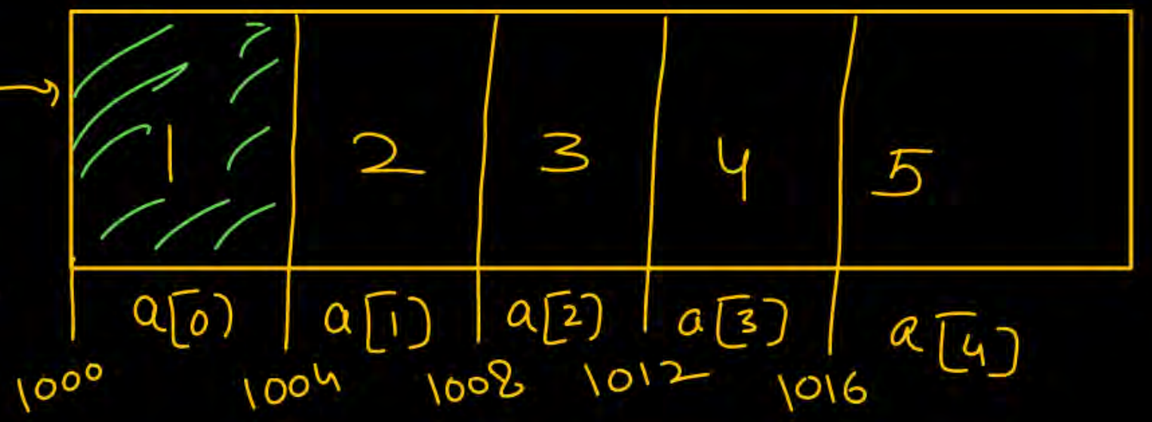
- `printf("%u", &a[0]+1);` `&a[0]+1*4 => 1000+4 = 1004`
- `printf("%u", &a+1);` `&a+1*16 = 1000+16 = 1016`

1:

1000
4 byte

a 1000

```
int a[5] = {1, 2, 3, 4, 5};
```



1) printf("%u", a); ⇒ a = &a[0] ⇒ 1000

2) printf("%u", &a); ⇒ 1000

3) printf("%u", *a); ⇒ ~~*&a[0]~~ ⇒ a[0] = 1

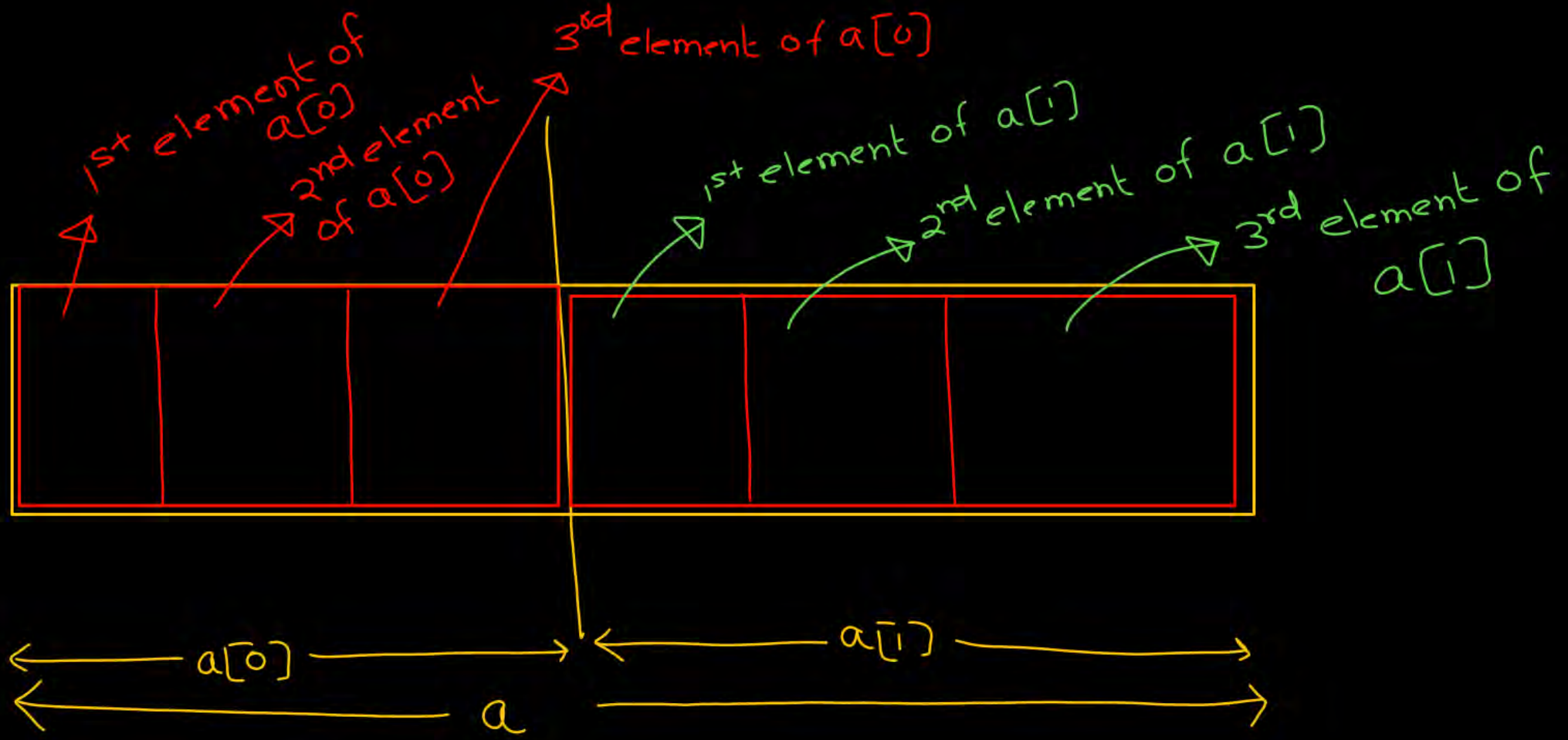
4) printf("%u", a+1); ⇒ &a[0] + 1 ⇒ &a[0] + 1 × 4 = 1000 + 4 = 1004
(4 byte)

5) printf("%u", &a+1); ⇒ &a + 1 ⇒ 1000 + 1 × 20 = 1020
(add the whole array)

6) printf("%u", *a+1); ⇒ 1+1 = (2)

2-D arrays

```
int a[2][3] = {1,2,3,4,5,6};
```

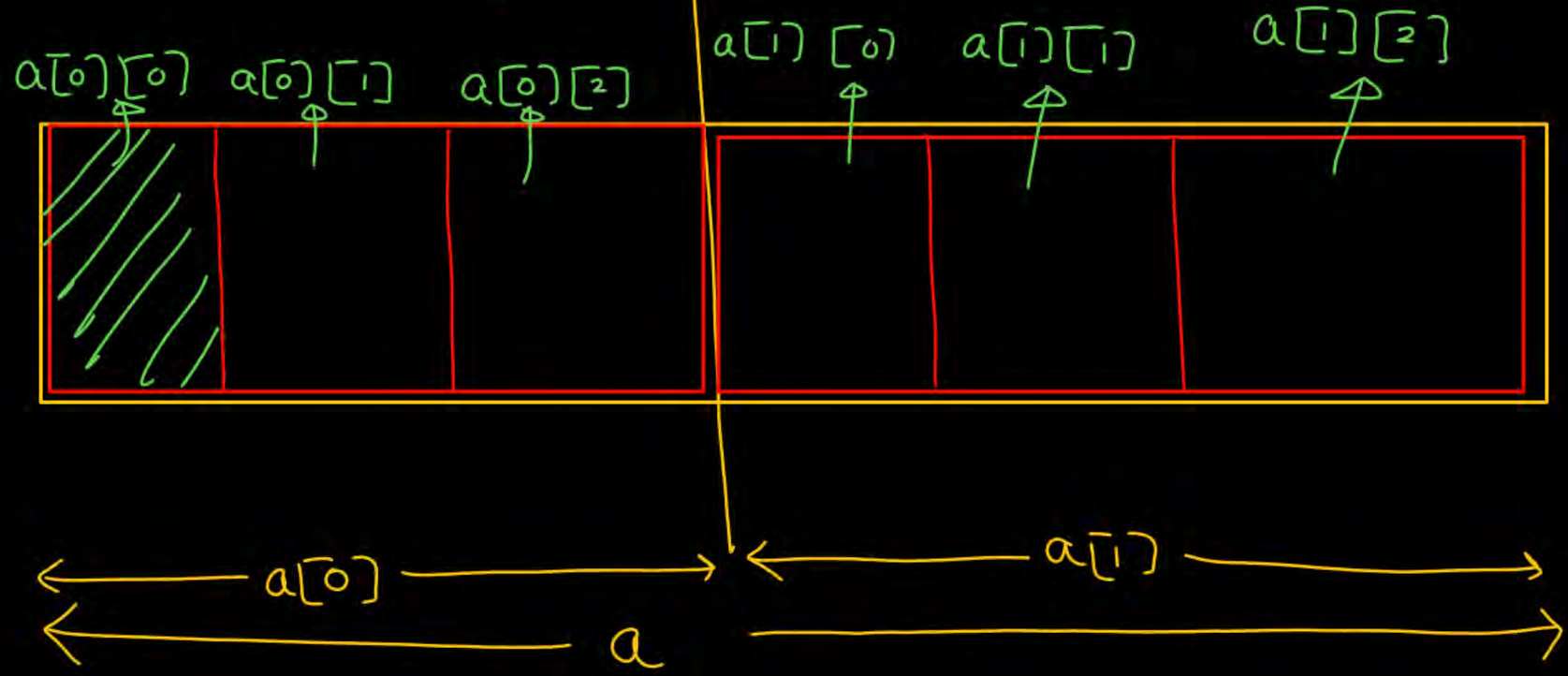


2-D arrays

```
int a[2][3] = {1,2,3,4,5,6};
```

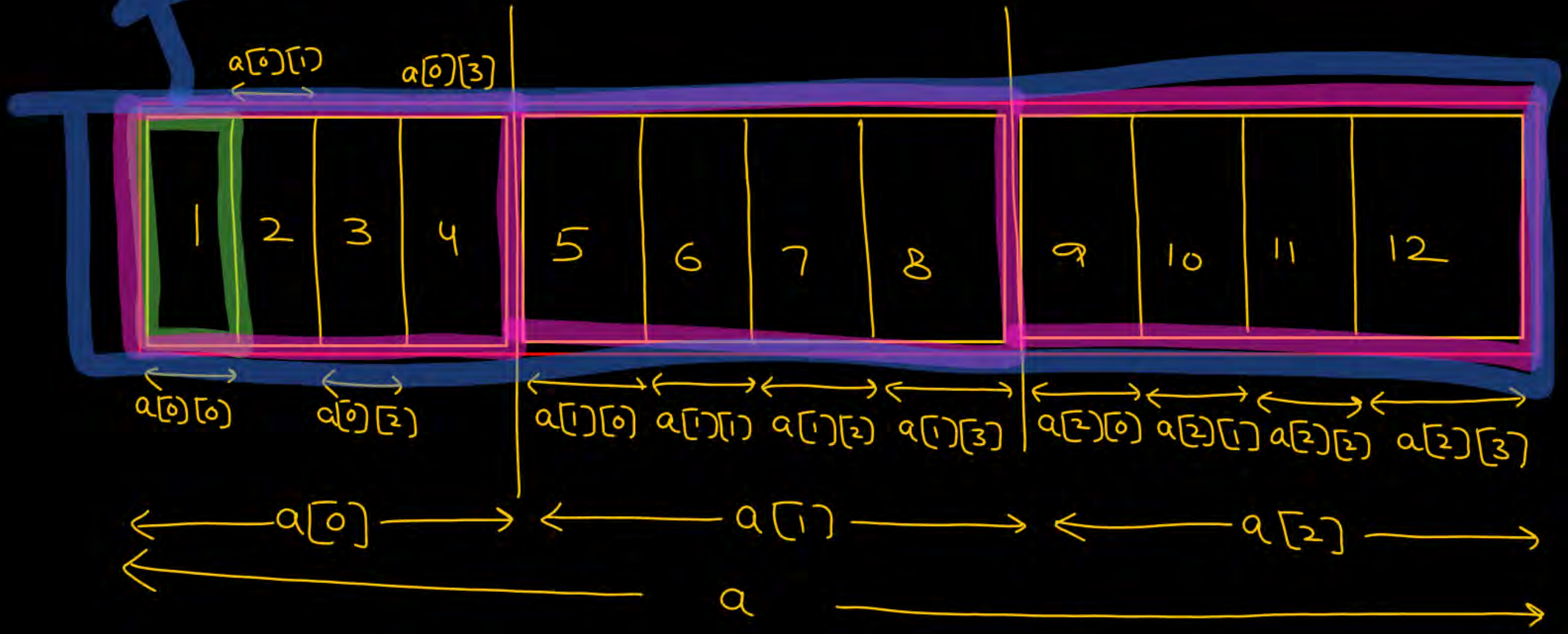
First element of a ⇒ a[0] ✓

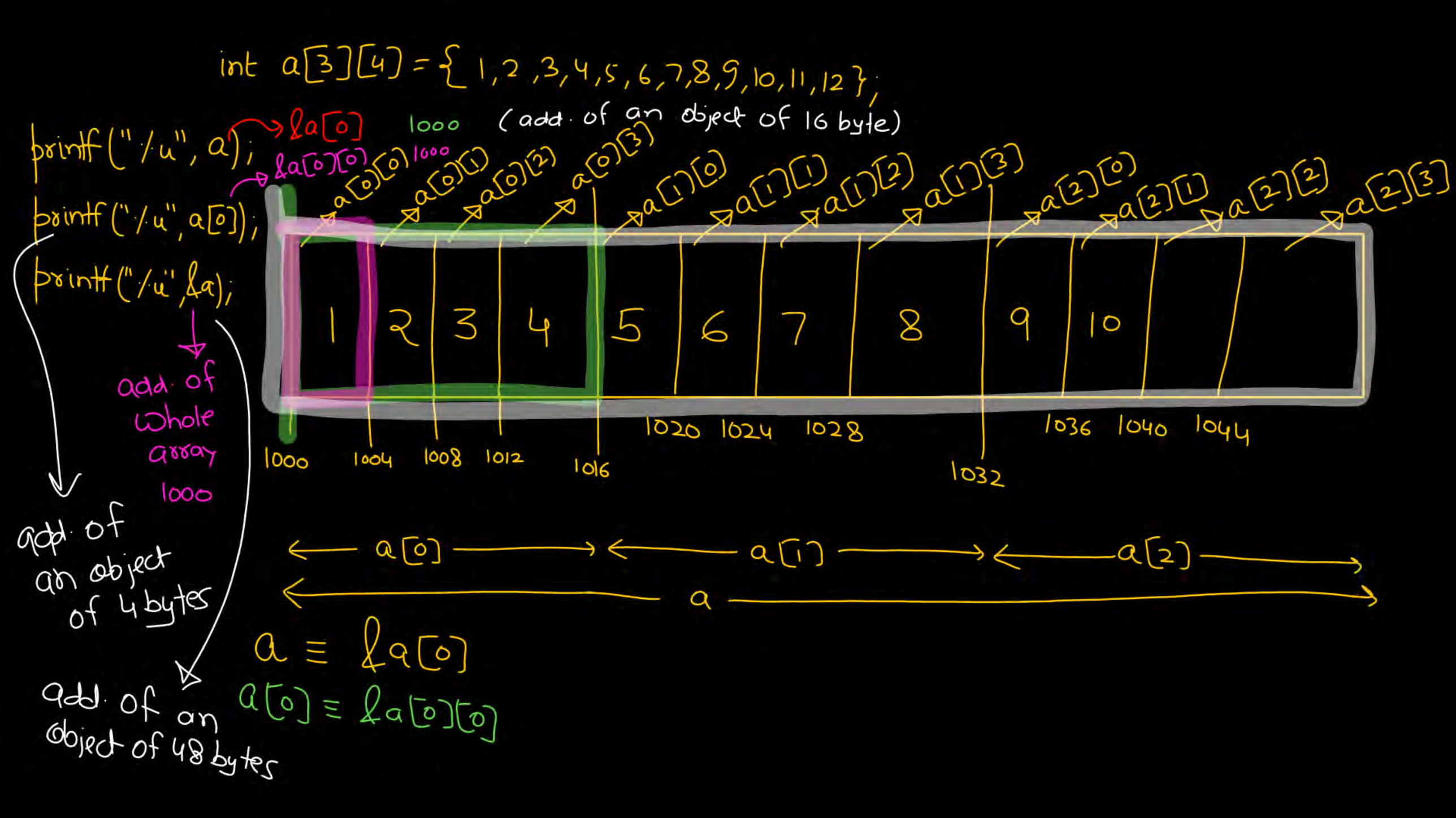
First element of a[0] ⇒ a[0][0] ✓



```
int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
```

48 bytes (whole array)




int a[3][4] = { 1,2,3,4,5,6,7,8,9,10,11,12 };
printf("%u", a);
printf("%u", a[0]);
printf("%u", &a);
&a[0]
&a[0][0]
1000
(add. of an object of 16 byte)
a[0][0] 1000
a[0][1]
a[0][2]
a[0][3]
a[1][0]
a[1][1]
a[1][2]
a[1][3]
a[2][0]
a[2][1]
a[2][2]
a[2][3]
1 2 3 4 5 6 7 8 9 10
1000 1004 1008 1012 1016 1020 1024 1028 1032 1036 1040 1044
add. of Whole array 1000
add. of an object of 4 bytes
add. of an object of 48 bytes
a[0]
a[1]
a[2]
a
a ≡ &a[0]
a[0] ≡ &a[0][0]

1600
4 bytes

Q

```
int a[2][3] = {1,2,3,4,5,6};
```

12byte `a = &a[0]` 1000 — `printf("%u", a);`

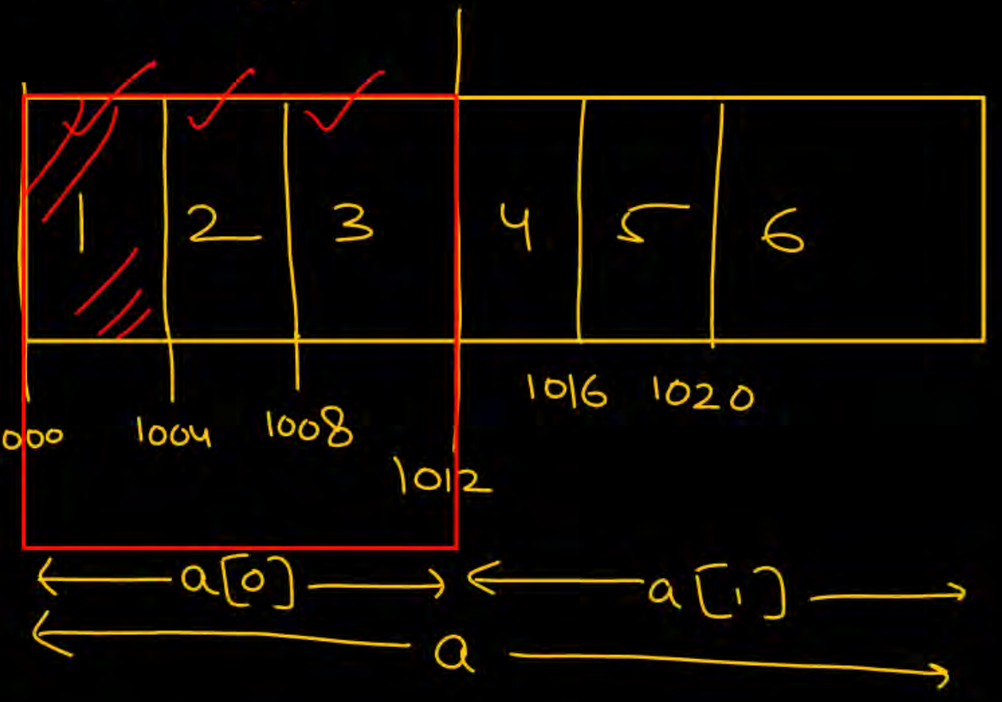
4byte `a[0] = &a[0][0]` 1000 — `printf("%u", a[0]);`

24byte `&a` ⇒ add of whole array 1000 — `printf("%u", &a);`

`a+1` ⇒ `&a[0]+1` = 1000 + 1×12 = 1012 — `printf("%u", a+1);`

`a[0]+1` = `&a[0][0]` + 1×4 = 1000 + 4 = 1004 — `printf("%u", a[0]+1);`

`&a+1` ⇒ `&a + 1×24` = 1000 + 24 = (1024) — `printf("%u", &a+1);`



1000
4 bytes

```
int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
```

printf("%u", a); → &a[0] ⇒ 1000

pf("%u", &a); → 1000

pf("%u", a[0]); → &a[0][0] ⇒ 1000

*a ≡ &a[0][0]

**a = *(&a[0][0])
= a[0][0]
= 1

pf("%u", a+1); → &a[0]+1 = 1000+16 = 1016

pf("%u", &a+1); → &a+1 ⇒ &a+1×48 = 1048

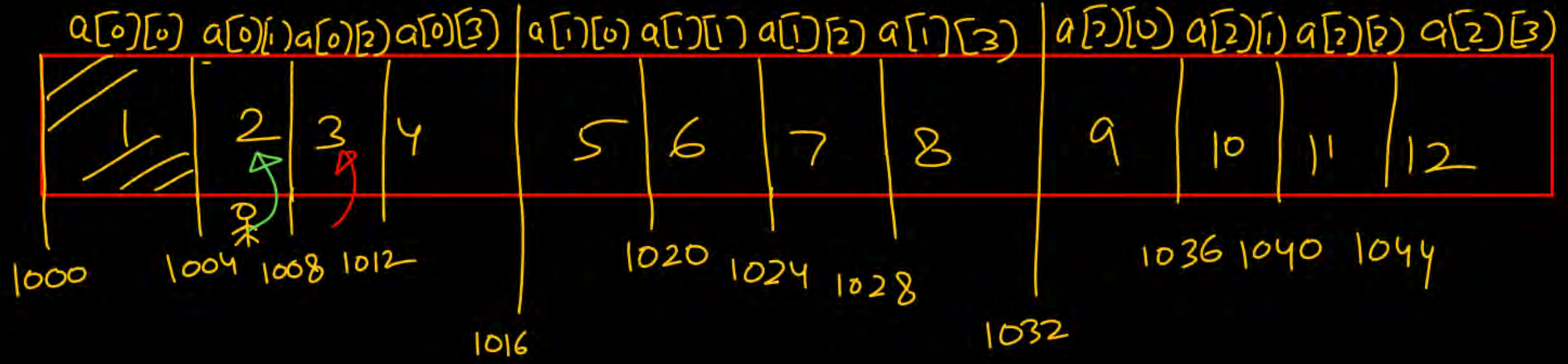
pf("%u", a[0]+1); → &a[0][0]+1 = 1000+1×4 = 1004

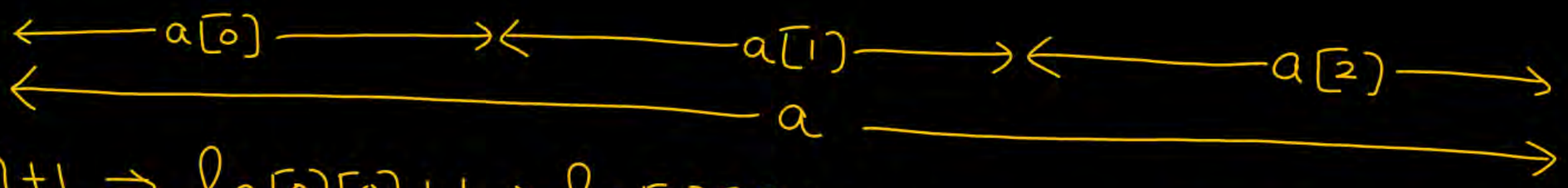
pf("%u", *a); → *a ⇒ *(&a[0]) ⇒ a[0] ⇒ &a[0][0] 1000

pf("%u", *a+1); → &a[0][0]+1 ⇒ 1000+1×4 = 1004

pf("%u", **a);

int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};





(i) a[0]+1 ⇒ &a[0][0]+1 ⇒ &a[0][0]+1×4 = 1004 (ii) a[0]+2 ⇒ &a[0][0]+2

(a[0]+1) ≡ Memory location 1004 = &a[0][1]

*(a[0]+1) ≡ Value at (Memory location 1004) = *&a[0][1]

*(a[0]+1) = 2 = a[0][1]

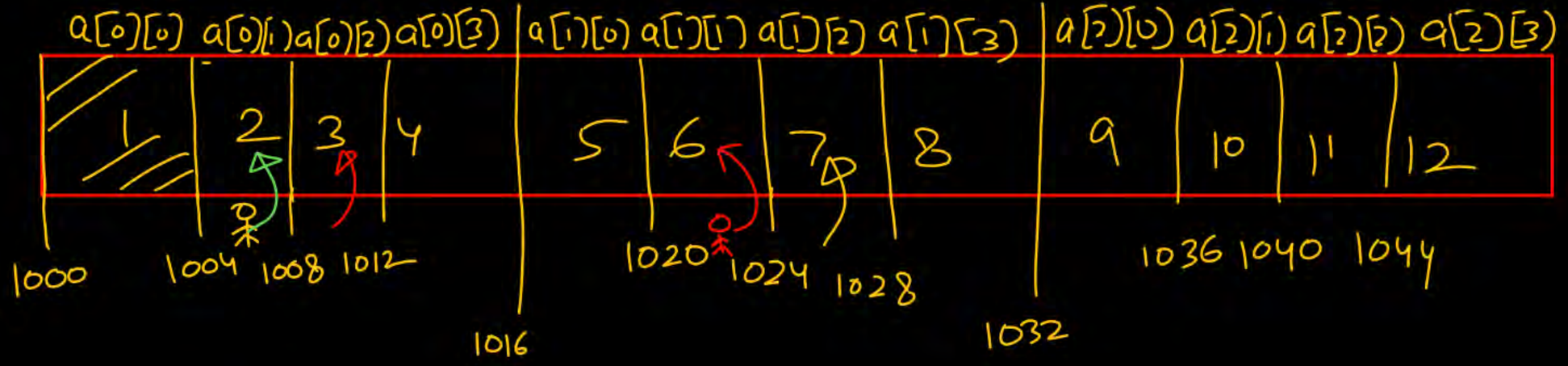
a[0]+2 = &a[0][0]+2×4 = 1008

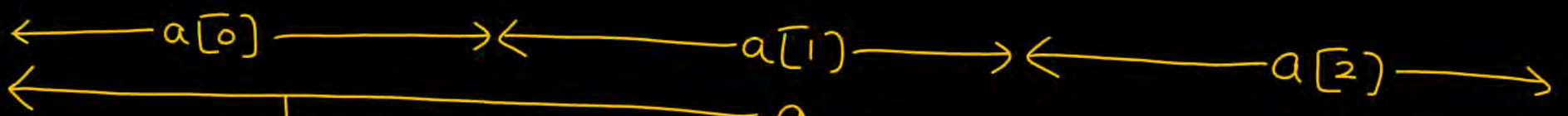
(a[0]+2) = Memory location 1008 = &a[0][2]

*(a[0]+2) = Value at (Mem. 1008) = *&a[0][2]

*(a[0]+2) = 3 = a[0][2]

int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};





*(a[0]+1) = 2 = a[0][1]

*(a[0]+2) = 3 = a[0][2]

*(a[0]+j) = a[0][j]

a[1]+1 ⇒ &a[1][0]+1 ⇒ &a[1][0]+1×4 = 1020

4 bytes

a[1]+1 ⇒ Memory loc. 1020 ⇒ &a[1][1]

*(a[1]+1) = value at (Mem. loc. 1020) = *&a[1][1]

*(a[1]+1) = 6 = a[1][1]

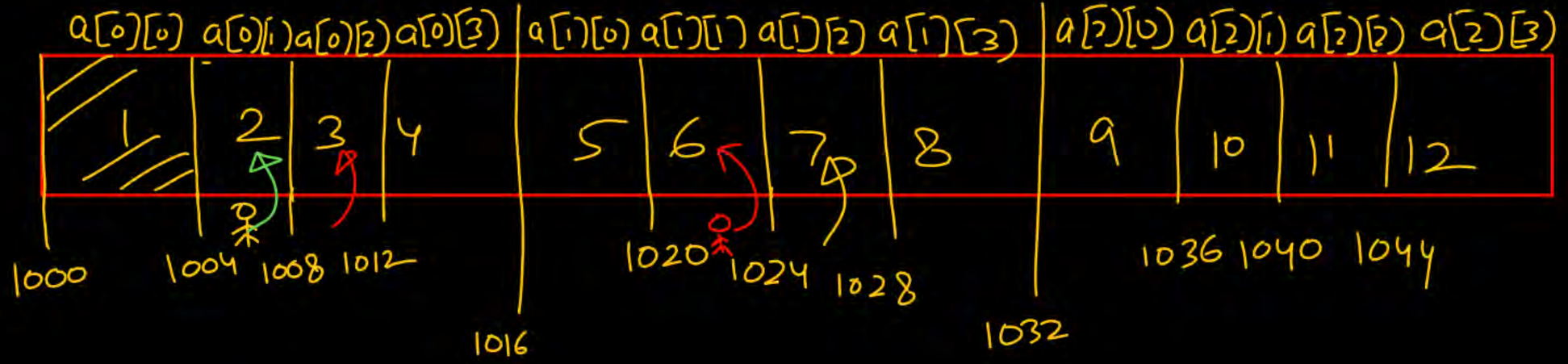
a[1]+2 ⇒ &a[1][0]+2×4

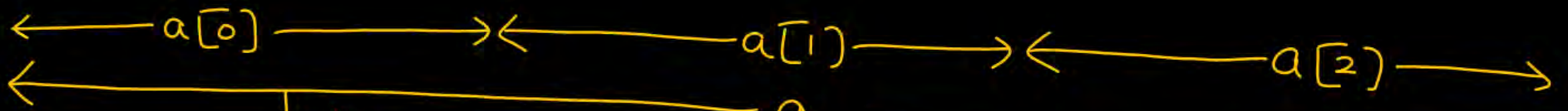
a[1]+2 = Mem. location 1024 = &a[1][2]

*(a[1]+2) = val at (Mem. loc. 1024) = *&a[1][2]

*(a[1]+2) = 7 = a[1][2]

int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};





*(a[0]+1) = 2 = a[0][1]

*(a[0]+2) = 3 = a[0][2]

*(a[0]+j) = a[0][j]

*(a[1]+1) = 6 = a[1][1]

*(a[1]+2) = 7 = a[1][2]

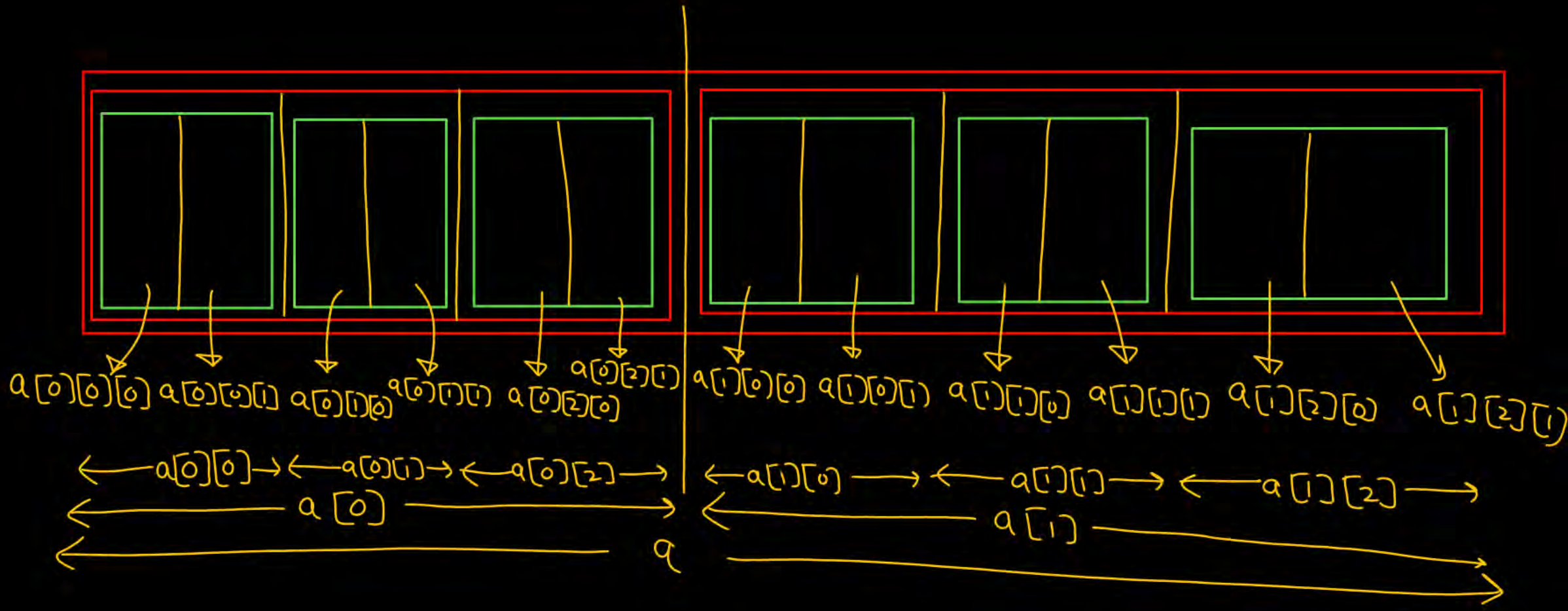
*(a[1]+j) = a[1][j]

*(a[i]+j) ≡ a[i][j]

≡ *(*(a+i)+j)

3-D array

```
int a[2][3][2] = {1,2,3,4,5,6,7,8,9,10,11,12};
```



1000
4 byte

```
int a[2][3][2] = {1,2,3,4,5,6,7,8,9,10,11,12};

pf("%u", a);        &a[0]          1000
pf("%u", &a);       whole array    1000
pf("%u", a[0]);     &a[0][0]       1000
pf("%u", a[0][0]);  &a[0][0][0]    1000
pf("%u", a+1);      &a[0]+1 => &a[0]+1x24   (1024)
pf("%u", &a+1);     &a+1 = &a+1x48 = (1048)
pf("%u", a[0]+1);   &a[0][0]+1 = &a[0][0]+1x8   (1008)
pf("%u", a[0][0]+1);
```

1004
&a[0][0][0]+1x4
&a[0][0][0]+1

```
pf("%u", *a);
pf("%u", **a);
pf("%u", ***a);
pf("%u", *a+1);
pf("%u", **a+1);
pf("%u", ***a+1);
```

*&a[0] = a[0] = &a[0][0]   (1000)

*a = &a[0][0]

**a = *&a[0][0] => a[0][0] => &a[0][0][0]   (1000)

***a = &a[0][0][0] = a[0][0][0] = 1

&a[0][0]+1 => 1008

&a[0][0][0]+1x4 = 1004

1+1 = 2

① If we are declaring an array without initialization then it is mandatory to provide size of each dimension.

```
int a[];          X
int a[4];         ✓
int a[][];        X
int a[2][];       X
int a[][3];       X
int a[2][3];      ✓
```

```
int a[][][];      X
int a[2][][];     X
int a[2][3][];    X
int a[][2][3];    X
int a[2][][3];    X
int a[2][3][3];   ✓
```

(2) If we are declaring an array with initialization, then there is a flexibility ⟹ Only for first dimension

- provide size
- or don't provide the size

✓ `int a[] = {1,2,3};`

`int a[][3] = {1,2,3,4,5,6};` ✓

`int a[2][3] = {1,2,3,4,5,6};` ✓

`int a[2][] = {1,2,3,4,5,6};` ✗

`int a[2][2][2] = {1,2,3,4,5,6,7,8};`

✗ `int a[][2][2] = {1,2,3,4,5,6,7,8};`

✗ `int a[2][][2] = {1,2,3,4,5,6,7,8};`

✗ `int a[2][2][] = {1,2,3,4,5,6,7,8};`


THANK YOU GW
SOLDIERS !